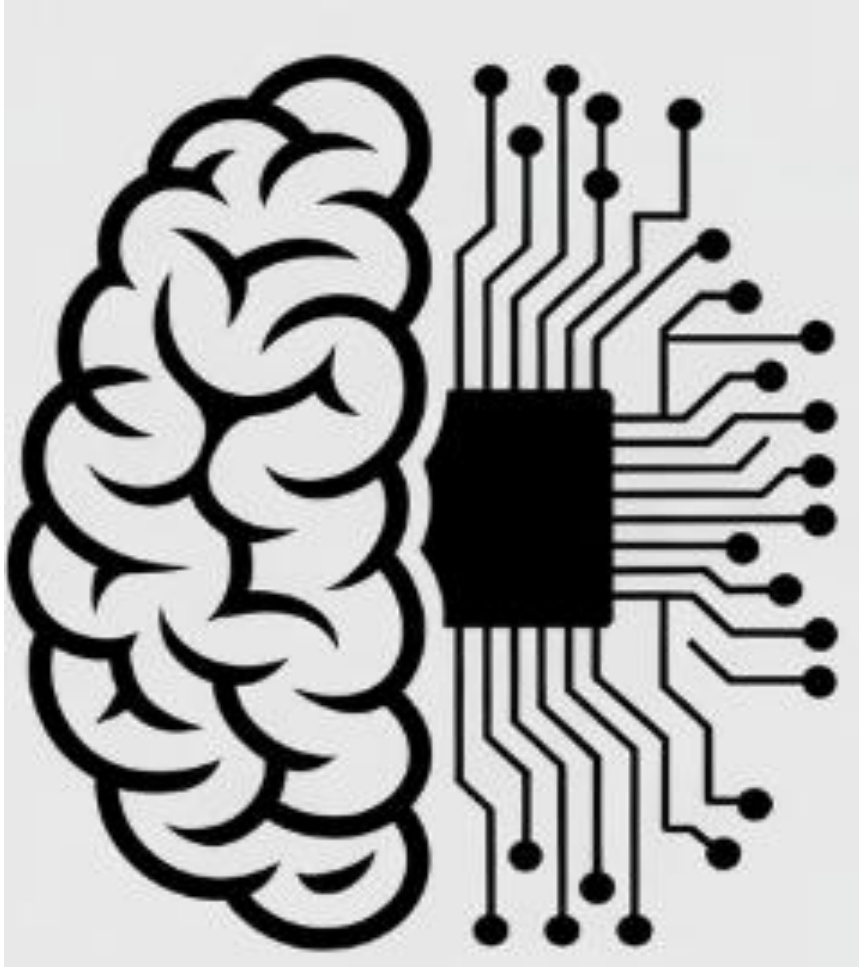
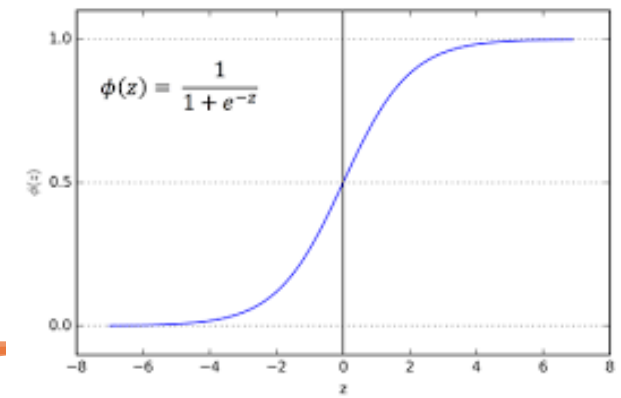




Neural Network Back Propagation

Dr. Sobia Tariq Javed

Sigmoid function



- $\sigma(x) = \frac{1}{1+e^{-x}}$
- $\frac{d\sigma(x)}{dx} = \frac{1}{1+e^{-x}}$
- $\frac{d\sigma(x)}{dx} = (1 + e^{-x})^{-1}$
- $\sigma'(x) = (-1)(1 + e^{-x})^{-2}(-1)(e^{-x})$
 $= \frac{e^{-x}}{(1+e^{-x})^2}$
 $= \left[\frac{1}{1+e^{-x}} \right] \left[\frac{e^{-x}}{1+e^{-x}} \right]$
 $= \left[\frac{1}{1+e^{-x}} \right] \left[\frac{-1+1+e^{-x}}{1+e^{-x}} \right]$

- $\sigma'(x) = \left[\frac{1}{1+e^{-x}} \right] \left[1 - \frac{1}{1+e^{-x}} \right]$
- Since $\sigma(x) = \frac{1}{1+e^{-x}}$
- $\sigma'(x) = \sigma(x)[1 - \sigma(x)]$

Activation functions

Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x) (1 - f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2 + 1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric Rectified Linear Unit (PReLU)		$f(x) = \begin{cases} ax & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} a & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU)		$f(x) = \begin{cases} a(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + a & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$

One-hot encoding

- One-hot encoding converts categorical variables into **binary vectors** where only one element is **1** and all others are **0**, ensuring no artificial ordering between categories.
- Example categorical data:
 - Color = {Red, Green, Blue}
- Models cannot directly understand these labels, so we encode them numerically without implying order.

	A ₁	A ₂	Color	Labels
X ₁	X ₁₁	X ₁₂	Red	L ₁
X ₂	X ₂₁	X ₂₂	Green	L ₂
X ₃	X ₃₁	X ₃₂	Green	L ₃
⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮
X _m	X _{m1}	X _{m2}	Blue	L _m

One-hot encoding

"Red" $\rightarrow [1, 0, 0]$

"Green" $\rightarrow [0, 1, 0]$

"Blue" $\rightarrow [0, 0, 1]$

- Number of unique values = 3
- Add 3 attributes and assign 0 or 1 based on the values.

	A ₁	A ₂	Color	Labels
X ₁	X ₁₁	X ₁₂	Red	L ₁
X ₂	X ₂₁	X ₂₂	Green	L ₂
X ₃	X ₃₁	X ₃₂	Green	L ₃
⋮	⋮	⋮	⋮	⋮
X _m	X _{m1}	X _{m2}	Blue	L _m



	A ₁	A ₂	Red	Green	Blue	Labels
X ₁	X ₁₁	X ₁₂	1	0	0	L ₁
X ₂	X ₂₁	X ₂₂	0	1	0	L ₂
X ₃	X ₃₁	X ₃₂	0	1	0	L ₃
⋮	⋮	⋮	⋮	⋮	⋮	⋮
X _m	X _{m1}	X _{m2}	0	0	1	L _m

One-hot encoding

- **Use when:**

- Features are categorical (nominal)
- No natural order exists

- **Limitations**

- High dimensionality for large categories
- Example: Country feature → 200+ dimensions
- Sparse vectors → memory inefficient

	A ₁	A ₂	Red	Green	Blue	Labels
X ₁	X ₁₁	X ₁₂	1	0	0	L ₁
X ₂	X ₂₁	X ₂₂	0	1	0	L ₂
X ₃	X ₃₁	X ₃₂	0	0	1	L ₃
⋮	⋮	⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮
X _m	X _{m1}	X _{m2}	0	0	1	L _m

Gradient Descent

- Gradient descent is an iterative method that **updates parameters** in the **direction of the negative gradient** to minimize the **loss function**.
- $W_i^{\text{new}} = W_i^{\text{old}} - \Delta W_i$
- $\Delta W_i = \text{learning rate} * \text{gradient of Error/loss function w.r.t weight}$
- $W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial E}{\partial W_i}$
- Its general optimization method (used in many algorithms).

Gradient Descent

- **Batch gradient descent (BGD)**
 - Uses the entire training dataset to compute the gradient before updating weights.
- **Stochastic gradient descent (SGD)**
 - Updates weights using one training example at a time
- **Mini-batch gradient descent (MGD)**
 - Compromise between BGD and SGD
 - Uses a small batch of samples (e.g., 32, 64)

Comparison

Mini-batch gradient descent is preferred because it balances speed, stability, and efficiency.

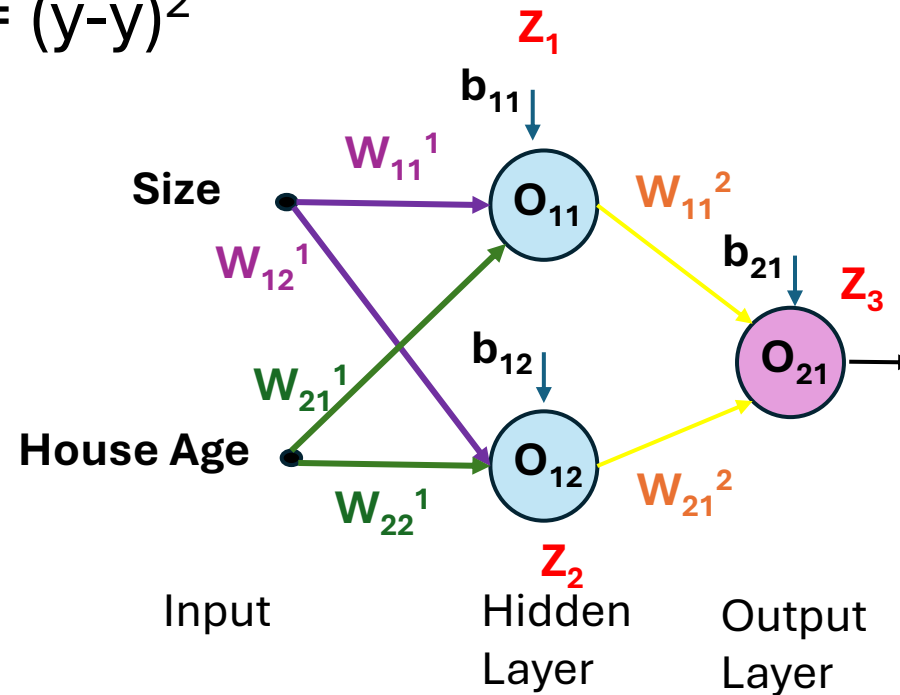
Feature	Batch GD	Mini-Batch GD	SGD
Data used	All data	Small batch	One sample
Speed	Slow	Medium	Fast
Stability	High	Medium	Low (noisy)
Memory	High	Medium	Low

Multi-layer NN- Regression

$$W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial L}{\partial W_i}$$

- Loss Function = $L = \text{MSE} = (y - \hat{y})^2$

	Size	House Age	Price
X_1	5	13.2	90
X_2	10	5	140
X_3	20	25	200
$\vdots$	$\vdots$	$\vdots$	$\vdots$
X_m	12	45	145



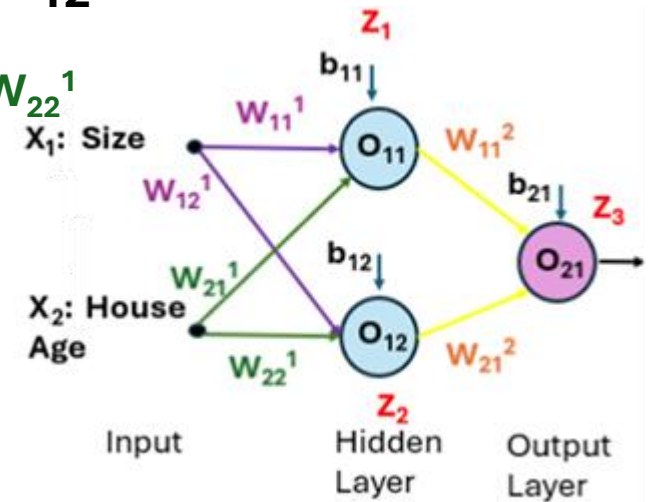
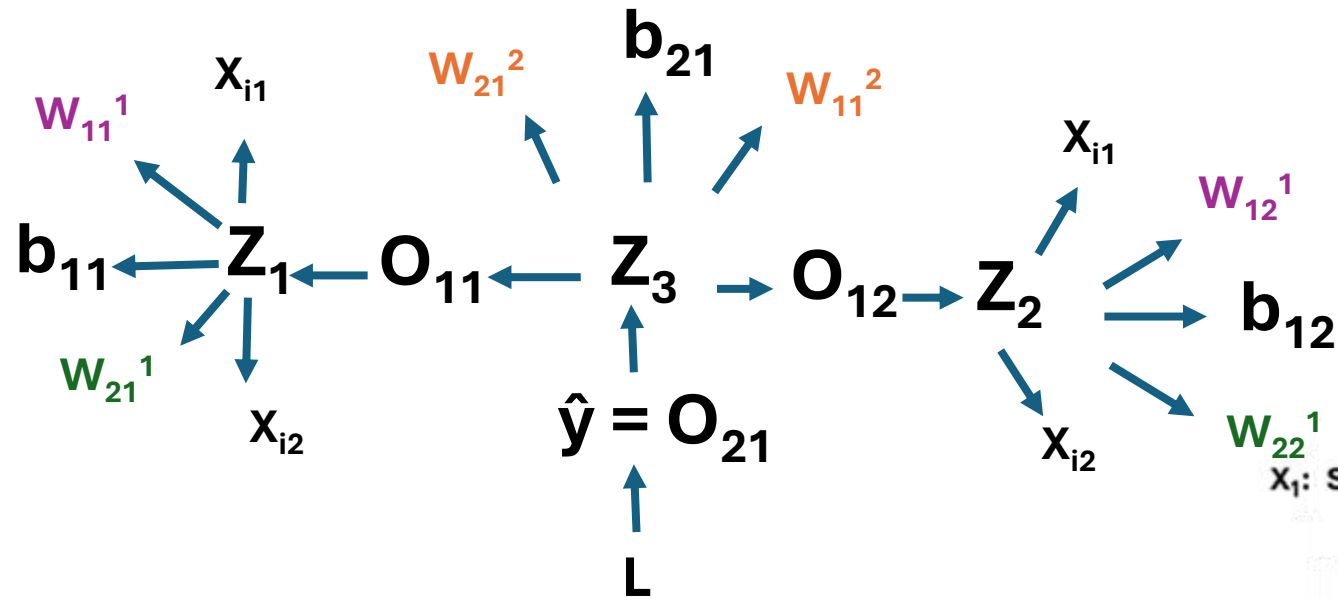
9 trainable parameters

Multi-layer NN- Regression

$$W_i^{\text{new}} = W_i^{\text{old}} - \eta * \frac{\partial L}{\partial W_i}$$

- Loss Function = $L = \text{MSE} = (y - \hat{y})^2$

	Size	House Age	Price
X_1	5	13.2	90
X_2	10	5	140
X_3	20	25	200
$\vdots$	$\vdots$	$\vdots$	$\vdots$
X_m	12	45	145



Multi-layer NN-Weight update

Training of Output Layer Weights

- L = Loss function

- General

$$\frac{\partial L}{\partial W_{ij}^{(l+1)}} = \frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial Z_{(l+1)j}} \cdot \frac{\partial Z_{(l+1)j}}{\partial W_{ij}^{(l+1)}}$$

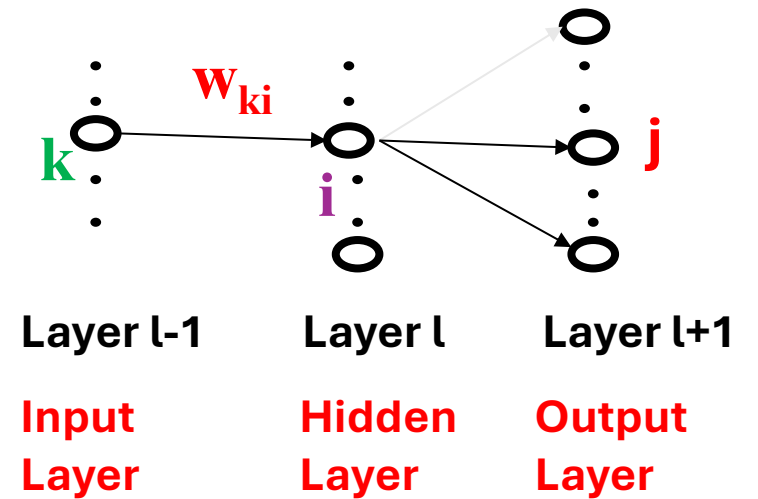
$$\frac{\partial L}{\partial W_{ij}^{(l+1)}} = c[\boxed{L'} \cdot \boxed{\sigma'(Z_{(l+1)j})} \cdot \boxed{X_{li}}]$$

L' = derivative of loss function

- $L' = 2(y - \hat{y})(-1) \rightarrow$ Regression Problem
- $L' = \frac{-(y - \hat{y})}{(1 - \hat{y})(\hat{y})} \rightarrow$ Classification Problem

$\sigma'(Z_{(l+1)j})$ = derivative of **activation function** at j th unit of output layer

$X_{li} = O_{li}$, output of the unit i of layer l



Multi-layer NN-Weight update

Training of Hidden Layer Weights

- L = Loss function

- General

$$\frac{\partial L}{\partial W_{ki}^{(l-1)}} = \sum_j \left(\frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial Z_{(l+1)j}} \cdot \frac{\partial Z_{(l+1)j}}{\partial O_{li}} \right) \cdot \frac{\partial O_{li}}{\partial Z_{li}^1} \cdot \frac{\partial Z_{li}^1}{\partial W_{ki}^1}$$

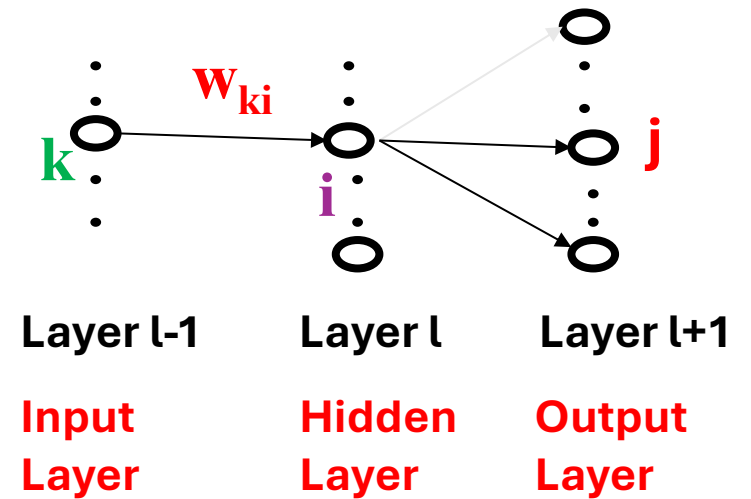
$$\frac{\partial L}{\partial W_{ki}^{(l-1)}} = c \sum_j [L' \cdot \sigma'(Z_{(l+1)j}) \cdot W_{ij}] \cdot \sigma'(Z_{li}) \cdot X_k$$

L' = derivative of loss function

- $L' = 2(y - \hat{y})(-1) \rightarrow$ Regression Problem
- $L' = \frac{-(y - \hat{y})}{(1 - \hat{y})(\hat{y})} \rightarrow$ Classification Problem

$\sigma'(Z_{(l+1)j})$ = derivative of **activation function** at j th unit of output layer
 W_{ij} = **Weight** of the unit i of layer l to unit j of layer $l+1$

$\sigma'(Z_{li})$ = derivative of **activation function** at i th unit of Hidden layer
 X_k = k th input of network.



Algorithm - Backpropagation

- **Input:**

- Training dataset (inputs with target outputs)
- η = Learning rate (e.g., 0.05)
- Multilayer feedforward neural network

- **Output:**

- A trained neural network (updated weights)

Algorithm - Backpropagation

- **Initialize the network**
 - Set all weights and biases to small random values (-0.05 to 0.05).
- **Until termination condition is met do**
 - **Step 1: Forward Pass**
 - Input the data into the network.
 - Compute outputs layer by layer (input $\rightarrow$ hidden $\rightarrow$ output).
 - Get the final predicted output.
 - **Step 2: Compute Error**
 - Compare predicted output with actual (target) output using loss function L.
 - $L = (y - \hat{y})^2 \rightarrow$ Regression Problem
 - $L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) \rightarrow$ Classification Problem

Algorithm - Backpropagation

- **Step 3: Backward Pass (Backpropagation)**

- Start from the output layer and move backward.

- For each output unit j calculate L' and $\sigma'(Z_{(l+1)j})$

- $Err_j = L' \cdot \sigma'(Z_{(l+1)j})$

$$\frac{\partial L}{\partial W_{ij}^{(l+1)}} = [\overset{E_j = Err_j}{L' \cdot \sigma'(Z_{(l+1)j})} \cdot X_{li}]$$

- For each unit i of the hidden layer from last to the first layer compute

- $Err_i = \sigma'(Z_{li}) \sum_j Err_j W_{ij}$ (Error w.r.t next layer j)

- For each weight in network

- $\Delta W_{ij} = \eta \cdot Err_i \cdot X_j$

- $W_{ij(new)} = W_{ij(old)} - \Delta W_{ij}$

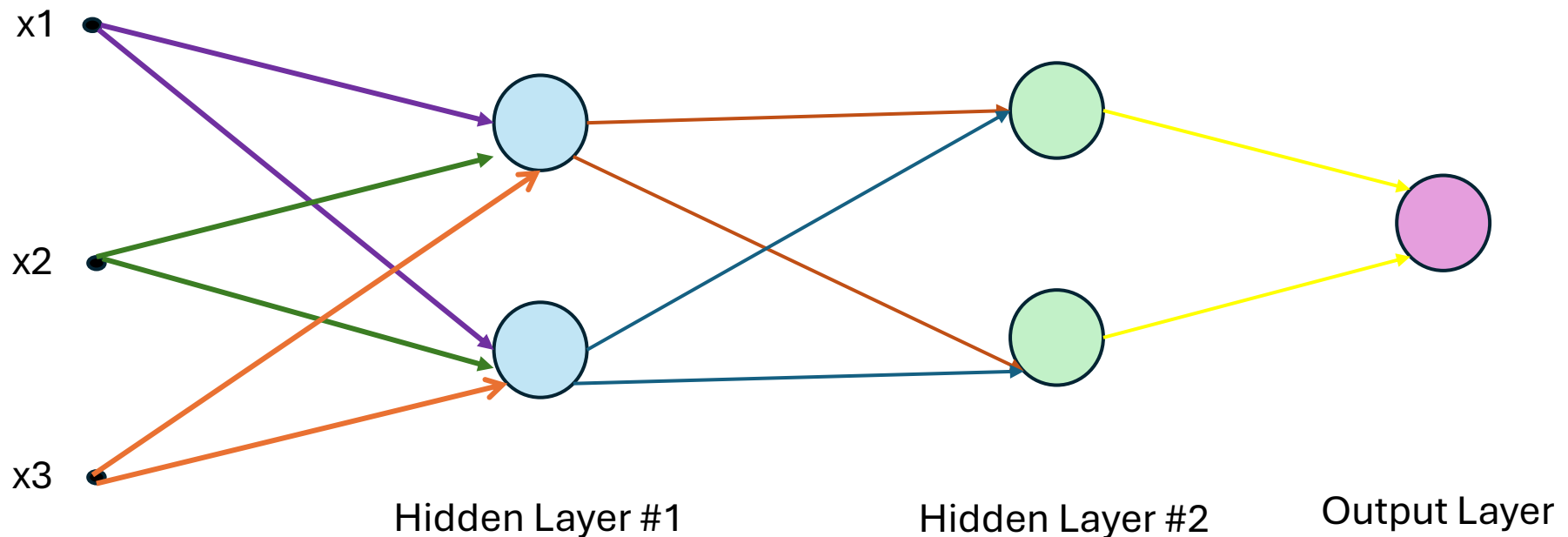
- $\Delta b_j = \eta \cdot Err_j$

- $b_{j(new)} = b_{j(old)} - \Delta b_j$

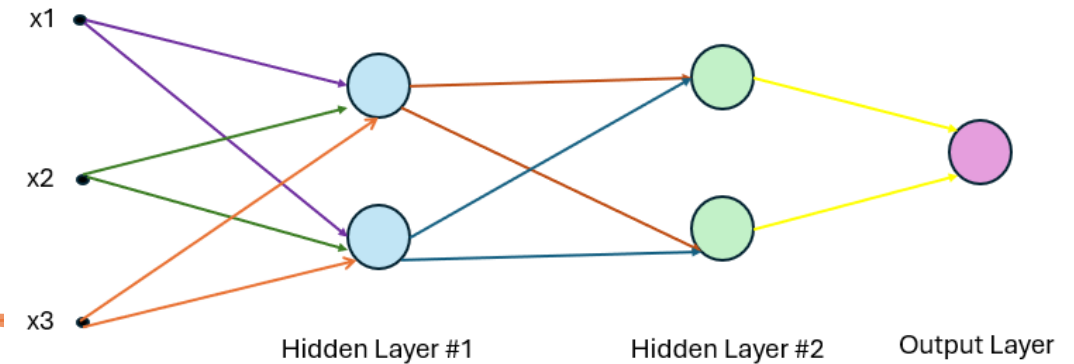
$$\frac{\partial L}{\partial W_{ki}^{(l-1)}} = \sum_j \overset{E_j = Err_j}{[L' \cdot \sigma'(Z_{(l+1)j}) \cdot W_{ij}]} \underset{E_i = Err_i}{\sigma'(Z_{li})} \cdot X_k$$

CW

- Apply the Forward + backward pass and label the network clearly according to the conventions covered in class.



CW



- $Y = 1$ (Actual output), $X1 = 1$, $X2 = 2$, $X3 = 1$
- $W_{11}^1 = 0.01$, $W_{12}^1 = 0.03$, $W_{21}^1 = 0.02$, $W_{22}^1 = 0.06$, $W_{31}^1 = 0.02$, $W_{32}^1 = 0.01$
- $b_{11} = b_{12} = b_{21} = b_{22} = -1$, $b_{31} = +1$
- $W_{11}^2 = 0.07$, $W_{12}^2 = 0.06$, $W_{21}^2 = 0.05$, $W_{22}^2 = 0.03$
- $W_{11}^3 = 0.06$, $W_{21}^3 = 0.01$

- Sigmoid

$$f(x) = \frac{1}{1 + \exp(-x)}; f'(x) = f(x)[1 - f(x)]$$

•

- Tanh

$$f(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}; f'(x) = [1 + f(x)][1 - f(x)]$$

$$Z_1 = \begin{bmatrix} -0.93 \\ -0.84 \end{bmatrix} \quad \begin{bmatrix} O_{11} \\ O_{12} \end{bmatrix} = \begin{bmatrix} -0.73 \\ -0.68 \end{bmatrix}$$

$$Z_2 = \begin{bmatrix} -1.085 \\ -1.065 \end{bmatrix} \quad \begin{bmatrix} O_{21} \\ O_{22} \end{bmatrix} = \begin{bmatrix} -0.79 \\ -0.78 \end{bmatrix}$$

$$Z_3 = 0.94 \quad \begin{bmatrix} O_{31} \end{bmatrix} = \begin{bmatrix} 0.719 \end{bmatrix}$$

- Loss function = $\frac{1}{2} (y - \hat{y})^2$

$$y = 1, \hat{y} = 0.719$$

- Compute Loss L

- $L = \frac{1}{2} (1 - 0.719)^2$

- $L = 0.03948$

- $L' = -(y - \hat{y}), \sigma' = [O_{31}(1 - O_{31})]$

- $E_{31} = -(y - \hat{y}) [O_{31}(1 - O_{31})]$

- $E_{31} = -0.056$

- $E_{22} = -0.0002$

- $E_{11} = -0.000434$

- $E_{12} = -0.000355$

$$\text{Err}_j = L' \cdot \sigma'(Z_{(l+1)j})$$

$$W_i^{\text{new}} = W_i^{\text{old}} + \Delta W_i$$

- $\Delta W_{11}^3 = ?, \Delta W_{21}^3 = ?$

-

- $\Delta W_{11}^3 = \eta E_{31} \cdot O_{21} = 0.030968$

- $W_{11}^3 = 0.06 - 0.030968 = 0.029$

- $\Delta W_{21}^3 = -0.020576$

- $b_{31} = 1 - 0.7 \times (-0.056) = 1.0392$

- $W_{11}^1 = W_{11}^1_{(old)} - \eta E_{11} \cdot X_1 = 0.0103038$

- $W_{22}^2 = W_{22}^2_{(old)} - \eta E_{22} \cdot O_{12} = 0.0299048$

$$\Delta W_{ij} = \eta \cdot \text{Err}_i \cdot X_i$$

$$W_{ij(new)} = W_{ij(old)} - \Delta W_{ij}$$

$$\Delta b_j = \eta \cdot \text{Err}_j$$

$$b_{j(new)} = b_{j(old)} - \Delta b_j$$

$$\eta = 0.7$$